End-to-End NLP Project Report

Name	**Mahmut Poyraz**
Course	**IYD 328 — Work Experience**
Project	**NLP Project: Sentiment Analysis Dashboard**
Dataset	**Amazon Reviews (Kaggle — bittlingmayer)**
Date	**June 19, 2026**
Models	**Logistic Regression · Naive Bayes · SVM**

This report documents the complete development of an end-to-end NLP sentiment analysis system, covering dataset preparation and preprocessing, machine learning model training and comparative evaluation, relational database integration, and the deployment of an interactive web-based dashboard for real-time inference and analytical visualization.

Table of Contents

1 Project Overview

This project presents a comprehensive, end-to-end Natural Language Processing (NLP) pipeline engineered for the automated sentiment classification of product reviews. The underlying architecture integrates diverse machine learning models, a SQL database infrastructure, and an interactive web-based dashboard to facilitate real-time sentiment inference and statistical visualization.

1.1 Objectives

- Build a complete NLP preprocessing pipeline covering text cleaning, normalization, tokenization, stopword removal, and TF-IDF feature extraction.
- Train and evaluate three machine learning classifiers: Logistic Regression, Naive Bayes, and Support Vector Machine.
- Implement a SQLite database to persistently store review texts, predicted sentiments, confidence scores, model names, and classification timestamps.
- Develop an interactive Streamlit dashboard enabling real-time sentiment analysis, trend visualization, and model comparison.
- Maintain a clean version-controlled Git repository with meaningful commit history throughout the development lifecycle.

1.2 System Architecture

The system follows a layered architecture where each component has a single responsibility. Data flows from the raw text files through the preprocessing pipeline, into the model training module, and is ultimately surfaced through the interactive dashboard.

LAYER	MODEL	DUTY
Data	data/	Raw .ft.txt files from Kaggle
Preprocessing	src/preprocess.py	Text cleaning, TF-IDF vectorization
Training	src/train.py	Model fitting, .pkl serialization
Evaluation	Src/evaluate.py	Metrics, confusion matrix generation
Storage	src/database.py	SQLite schema
Interface	dashboard/app.py	Streamlit dashboard

2 Dataset

The dataset used in this project is the Amazon Reviews corpus published on Kaggle under the identifier bittlingmayer/amazonreviews. It contains real product reviews collected from Amazon spanning all product categories, stored in FastText binary classification format where each line begins with a sentiment label followed by the review text. The label mapping applied throughout this project designates __label__2 as Positive and __label__1 as Negative. As the dataset contains binary labels exclusively, this project implements binary sentiment classification without a neutral category.

3 NLP Processing Pipeline

AllAll raw text data passes through a multi-stage preprocessing pipeline implemented in src/preprocess.py prior to model training. The pipeline is applied identically to both training and test sets to prevent data leakage and ensure consistent feature representation across all splits.

3.1 Text Cleaning

- Lowercasing: All text is converted to lowercase to treat "Product" and "product" as the same token.
- HTML tag removal: Any residual HTML markup **is stripped using regex.**
- **URL removal: Web addresses are removed as they carry no sentiment signal.**
- **Punctuation removal: All punctuation characters are stripped using str.translate(). • Digit removal: Numeric tokens are removed to reduce noise.**
- **Whitespace normalization: Multiple consecutive spaces are collapsed into one.**

3.2 Tokenization and Stopword Removal

Tokenization is performed using NLTK's word_tokenize function, which segments text into individual tokens while correctly handling contractions and hyphenated expressions. English stopwords from NLTK's stopwords corpus are subsequently removed, as terms such as "the", "is", and "at" carry negligible semantic signal and their elimination reduces the feature space without meaningful information loss. Tokens shorter than two characters are additionally discarded.

4. Machine Learning Models

Three machine learning classifiers were selected to represent a range of algorithmic approaches, from probabilistic baselines to discriminative linear models. All models were trained on an identical TF-IDF feature matrix to ensure a fair and consistent basis for comparison.

4.1 Logistic Regression

Logistic Regression is a linear discriminative classifier that models the probability of class membership using a sigmoid activation over a weighted linear combination of input features. It is particularly well-suited to high-dimensional sparse TF-IDF feature spaces because it learns a direct mapping from word importance weights to sentiment probability. The SAGA solver was used to enable efficient optimization on large datasets. This model achieved the highest F1-Score of 90.37% and was selected as the default model in the dashboard.

4.2 Naive Bayes

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' theorem with the naive assumption that features are conditionally independent given the class. Despite this strong assumption, it performs remarkably well on text classification tasks and serves as an efficient, interpretable baseline. Its training time is significantly shorter than the other models. The smoothing parameter alpha=0.1 was used to handle zero-frequency terms.

4.3 Support Vector Machine

LinearSVC identifies the optimal separating hyperplane that maximizes the margin between the two sentiment classes within the TF-IDF feature space, making it one of the strongest performers on high-dimensional text data. As LinearSVC does not natively produce probability estimates, it was wrapped with CalibratedClassifierCV to enable confidence score computation through Platt scaling, a requirement for confidence visualization within the dashboard.

5 Evaluation Results

5.1 Performance Metrics

All models were evaluated on a held-out test set of 40,000 samples that were not seen during training. Four standard classification metrics were computed:

• Accuracy: Proportion of correctly classified samples out of all samples.

• Precision: Of all samples predicted as Positive, what fraction were actually Positive.

• Recall: Of all actual Positive samples, what fraction were correctly identified.

• F1-Score: Harmonic mean of Precision and Recall — the primary ranking metric.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	90.15%	89.92%	90.83%	90.37%
SVM	90.10%	89.93%	90.69%	90.31%
Naive Bayes	87.71%	88.07%	87.74%	87.90%

Logistic Regression and SVM perform nearly identically, with Logistic Regression marginally ahead on both Recall and F1-Score. The 2.5 percentage point gap between these two models and Naive Bayes reflects the superior ability of discriminative models to exploit the full TF-IDF feature space without independence assumptions. All three models comfortably exceed 87% accuracy on the test set, demonstrating that the preprocessing pipeline and feature extraction strategy are effective.

5.2 Confusion Matrix Analysis

Confusion matrices were generated for all three models on the 40,000 sample test set. The matrices are saved as PNG files in the models/ directory and are displayed interactively in the dashboard's Model Comparison tab.

	Predicted Positive	Predicted Negative
Actual Positive	18,492	1,867
Actual Negative	2,057	17,584

The model correctly classifies 18,492 positive and 17,584 negative reviews. False positives (1,867) and false negatives (2,057) are roughly symmetric, indicating balanced model behavior with no significant directional bias.

6. Dashboard

The dashboard is built with Streamlit and structured into four functional tabs, each serving a distinct analytical purpose. It connects directly to the trained machine learning models and the SQLite database, delivering real-time sentiment classification alongside persistent, queryable analytics. The interface follows a clean light theme designed for clarity and professional presentation.

6.1 Analyze Review

The core classification interface through which users submit product reviews for sentiment prediction. Upon submission, the selected model preprocesses and vectorizes the input text using the persisted TF-IDF vectorizer, returning a Positive or Negative classification accompanied by a confidence score and an animated progress bar. Pre-loaded example reviews are available for immediate evaluation. Every classification result, including the review text, predicted sentiment, confidence score, model name, and timestamp, is automatically written to the database.

6.2 Statistics and Trends

Provides an aggregated analytical view of all reviews classified through the dashboard. The tab presents key performance indicators covering total review count, positive and negative breakdowns, and the positive-to-negative ratio. A donut chart visualizes the overall sentiment distribution, while a timeline chart tracks classification activity over time. Horizontal frequency bar charts display the most prominent words within each sentiment class, and a word cloud offers an intuitive, filterable representation of vocabulary patterns across Positive, Negative, or combined review sets.

6.3 Model Comparison

Enables a structured, evidence-based comparison of all three trained models. A grouped bar chart presents Accuracy, Precision, Recall, and F1-Score side by side for each classifier. Confusion matrices are rendered as images to illustrate true positive, true negative, false positive, and false negative distributions. A radar chart provides a unified capability overview across all four metrics simultaneously. A summary table consolidates all numerical results for direct reference.

6.4 Review History

A complete, filterable record of every review stored in the database. Users can narrow results by sentiment class or by the model used for classification. Each record displays the original review text, predicted sentiment, confidence score, model name, and classification timestamp, providing full auditability of the system's inference history.

7. Conclusion

This project presents a fully operational, end-to-end sentiment analysis system trained on 200,000 Amazon product reviews. The preprocessing pipeline covers text normalization, stopword removal, tokenization, and TF-IDF feature extraction, providing a consistent and reliable foundation for classification.

Three machine learning models were trained and evaluated on a held-out test set of 40,000 samples. Logistic Regression achieved the highest performance with 90.15% accuracy and an F1-Score of 90.37%, followed by the Support Vector Machine at 90.10% accuracy. Multinomial Naive Bayes delivered a competitive baseline at 87.71%. These results confirm that classical linear models, when combined with well-engineered TF-IDF representations, can achieve production-relevant accuracy without the overhead of deep learning architectures.

The interactive dashboard serves as the primary user-facing layer of the system, enabling real-time sentiment inference with confidence scoring, sentiment trend visualization, word frequency analysis, and side-by-side model comparison. All classification outputs are automatically persisted to a SQLite database, ensuring that analytical insights accumulate and remain queryable over time.

The codebase follows modular design principles, with each component maintaining a single, clearly defined responsibility. Development was tracked through Git with semantically meaningful commit messages, providing full traceability of the project lifecycle from initial setup through final deployment.

Overall, this project demonstrates that a well-architected NLP system delivering strong predictive performance can be constructed entirely with open-source tools while maintaining clean, maintainable, and thoroughly documented code.